LAB 3:

CF Pipeline

Pipeline + Hyperparameter Tuning

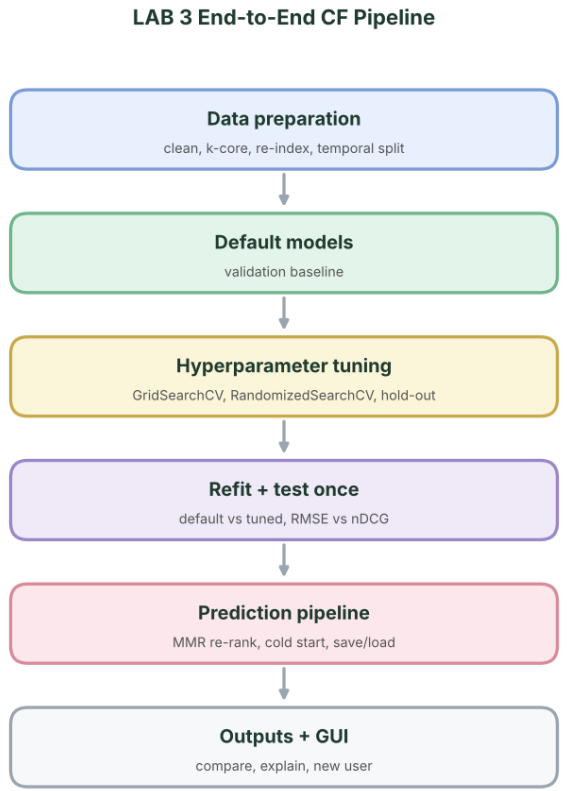
204729

Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

Prep - Tuning - Prediction

Evaluation - GUI



อธิบายแนวคิดของ Source Code ทีละ Part

GridSearchCV - RMSE vs nDCG - MMR - Cold start - Gradio

COMPUTER SCIENCE
Chiang Mai University

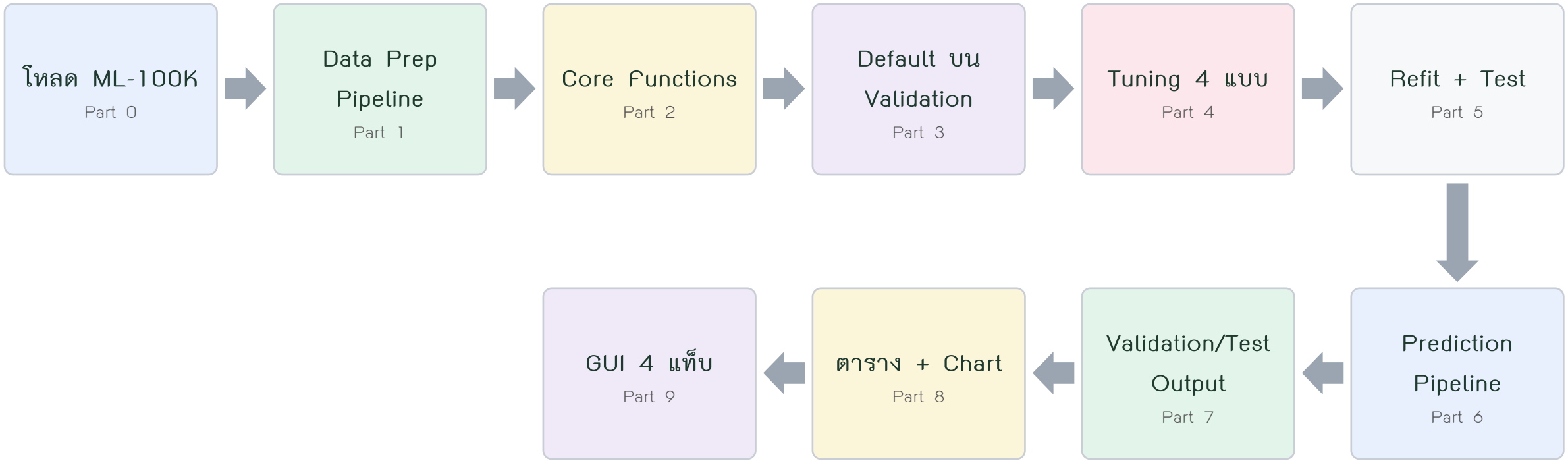
LAB 3 คืออะไร

ภาพรวม

- ต่อยอด LAB 2 (MovieLens-100K + Surprise) เป็น **Pipeline ครบวงจร** ตามแนวปฏิบัติในหัวข้อ 5.16
- **เตรียมข้อมูล** เป็นขั้น ๆ พร้อม Data Card: clean ⟶ 5-core ⟶ re-index (CSR) ⟶ split ตามเวลา 80/10/10 ⟶ ตรวจ leakage
- **ปรับ Hyperparameter** สามวิธี: GridSearchCV (SVD) • RandomizedSearchCV (KNNBaseline) • Hold-out บน Validation ที่วัดทั้ง RMSE และ nDCG@10
- **Refit** บน Train+Validation แล้วประเมิน **ครั้งเดียว** บน Test: Default vs Tuned ทั้ง Accuracy และ Beyond-Accuracy
- **Prediction Pipeline:** ranker + rater, กรองเรื่องที่ดูแล้ว, MMR re-ranking, cold-start fallback และ save/load ด้วย pickle
- ตัวอย่าง Output บน Validation/Test และ **GUI 4 แท็บ:** ผู้ใช้เดิม • เทียบโมเดล • อธิบายค่าทำนาย • ผู้ใช้ใหม่

ไฟล์: 03_Lab_CF_MovieLens_Pipeline_Tuning_Prediction_GUI.ipynb • รันบน Colab ราว 10–15 นาที

โครงสร้าง Notebook LAB 3 (Part 0 → Part 9)



กฎสำคัญ: ทุกการตัดสินใจใช้ Validation • Test ถูกเปิดครั้งเดียวใน Part 5

แผนที่ของ Source Code LAB 3

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ผลลัพธ์
0	โหลดข้อมูล	`load_movielens_100k` (ชุดเดียวกับ LAB 2)	ratings / items / users
1	เตรียมข้อมูล	`step_clean`, `step_kcore`, `step_reindex`, `step_split`, `DATA_CARD`	train / valid / test
2	Core Functions	LAB 2 + `evaluate_rating_df`, `evaluate_lists`	เครื่องมือประเมิน
3	ระบบฐาน	`DEFAULT_FACTORIES`, `default_valid`	8 โมเดลบน Validation
4	ปรับจูน	`GridSearchCV`, `RandomizedSearchCV`, `tune`, `best_by`, `curve`	config ที่ดีที่สุด
5	ทดสอบครั้งเดียว	`FINAL_FACTORIES`, `final_results`	Default vs Tuned บน Test
6	Prediction Pipeline	`RecPipeline`, `div_results`, `rec_pipeline.pkl`	พร้อมใช้งาน
7–8	Output + สรุป	`SAMPLE_USERS`, `summary`, `gain`	ตาราง + chart
9	GUI	`gui_user`, `gui_compare`, `gui_explain`, `gui_newuser`	แอป 4 แท็บ

Part 0: Environment และ Dataset

```
!pip -q install scikit-surprise gradio

import math, random, warnings, itertools, time, re, pickle
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.metrics import roc_auc_score

from surprise import (Dataset, Reader, accuracy,
    NormalPredictor, BaselineOnly, KNNBasic, KNNWithMeans,
                     KNNWithZScore, KNNBaseline, SVD,
                         SVDpp, NMF, SlopeOne,
                         CoClustering)
from surprise.model_selection import GridSearchCV,
    RandomizedSearchCV, cross_validate

warnings.filterwarnings("ignore")
pd.set_option("display.width", 220);
    pd.set_option("display.max_columns", 40)
SEED = 42
np.random.seed(SEED); random.seed(SEED)
```

อธิบายโค้ด

เหมือน LAB 2 + เครื่องมือปรับจูน

- เพิ่ม `GridSearchCV`, `RandomizedSearchCV` จาก `surprise.model_selection`
- `pickle` สำหรับบันทึก/โหลด Pipeline ใน Part 6
- ฟังก์ชัน `load_movielens_100k` ใช้ชุดเดียวกับ LAB 2 (มีทางสำรองจาก GitHub)
- `SEED = 42` ทุกขั้น ⟶ ผลทำซ้ำได้

Part 1: Data Card และ step clean

```
from scipy.sparse import csr_matrix
DATA_CARD = []
def log_step(step, df, note=""):
    nu, ni = df.user_id.nunique(),
        df.item_id.nunique()
    DATA_CARD.append({"step": step, "ratings":
        len(df), "users": nu, "items": ni,
                        "density": len(df) / (nu * ni)
                            if nu and ni else 0,
                            "note": note})
    return df

def step_clean(df):
    d = df.dropna(subset=["user_id", "item_id",
        "rating"])
    d = d[(d.rating >= 1) & (d.rating <= 5)]
    d = d.sort_values("timestamp").drop_duplicates(["u
        ser_id", "item_id"], keep="last")
    return log_step("1 clean", d, f"removed {len(df)
        - len(d)} rows")
```

อธิบายโค้ด

ทุกขั้นบันทึกลง Data Card

- `log_step` เก็บจำนวนคะแนน ผู้ใช้ รายการ ความหนาแน่น และหมายเหตุของแต่ละขั้น
- `step_clean`: ตัดค่าว่าง ตรวจช่วง 1–5 และกำจัดคะแนนซ้ำ (เก็บค่าล่าสุดตามเวลา)
- ML-100K สะอาดแล้ว → removed 0 rows แต่ code พร้อมใช้กับข้อมูลจริงที่สกปรก

Part 1: k-core และ Re-index

```python
def step_kcore(df, k_user=5, k_item=5):
    d = df.copy(); rounds = 0
    while True:
        rounds += 1
        uc = d.user_id.map(d.user_id.value_counts());
            ic = d.item_id.map(d.item_id.value_counts())
        keep = (uc >= k_user) & (ic >= k_item)
        if keep.all(): break
        d = d[keep]
    return log_step(f"2 {k_user}/{k_item}-core", d,
        f"{rounds} rounds")

def step_reindex(df):
    d = df.copy()
    d["uidx"] = d.user_id.astype("category").cat.codes;
        d["iidx"] =
        d.item_id.astype("category").cat.codes
    M = csr_matrix((d.rating.values, (d.uidx.values,
        d.iidx.values)))
    log_step("3 re-index (CSR)", d, f"CSR {M.shape},
        nnz={M.nnz}")
    return d, M
```

อธิบายโค้ด

ตัดข้อมูลบางและเตรียมเมทริกซ์

- k-core: ตัดผู้ใช้/รายการที่มี < k คะแนน **ซ้ำจนคงที่** (2 รอบ)
- 5-core ตัด 333 เรื่องเหลือ 1,349 เรื่อง (99,287 คะแนน) — ต้อง **รายงานค่า k** เสมอ
- Re-index: รหัส ⟶ 0..|U|−1, 0..|I|−1 ด้วย category codes
- สร้าง `csr_matrix` 943×1,349 (nnz 99,287) สำหรับระบบขนาดใหญ่

Part 1: แบ่งตามเวลา + ตรวจ Leakage

```python
def step_split(df, valid=0.1, test=0.1):
    d = df.sort_values(["user_id", "timestamp", "item_id"]).copy()
    d["rk"] = d.groupby("user_id").cumcount(); d["n"] =
        d.groupby("user_id")["item_id"].transform("size")
    frac = d.rk / d.n
    tr = d[frac < 1 - valid - test]; va = d[(frac >= 1 - valid -
        test) & (frac < 1 - test)]; te = d[frac >= 1 - test]
    for name, part in [("4 train", tr), ("4 validation", va), ("4
        test", te)]: log_step(name, part)
    return tr.drop(columns=["rk", "n"]), va.drop(columns=["rk",
        "n"]), te.drop(columns=["rk", "n"])

def step_leakage_check(tr, va, te):
    last_tr = tr.groupby("user_id").timestamp.max(); first_va =
        va.groupby("user_id").timestamp.min()
    first_te = te.groupby("user_id").timestamp.min(); last_va =
        va.groupby("user_id").timestamp.max()
    bad_va = int((first_va < last_tr.reindex(first_va.index)).sum())
    bad_te = int((first_te <
        last_va.reindex(first_te.index).fillna(last_tr.reindex(first
        _te.index))).sum())
    cold_va = len(set(va.item_id) - set(tr.item_id)); cold_te =
        len(set(te.item_id) - set(tr.item_id) - set(va.item_id))
    return pd.Series({"users with valid before train": bad_va,
        "users with test before valid": bad_te,
                      "cold-start items in valid": cold_va,
                          "cold-start items in test": cold_te})
```

อธิบายโค้ด

Train / Validation / Test ต่อผู้ใช้

- เรียงคะแนนของผู้ใช้ตามเวลา: 80% แรก ⟶ Train, 10% ถัดไป ⟶ Validation, 10% ล่าสุด ⟶ Test
- **step_leakage_check**: Validation ต้องเกิดหลัง Train และ Test หลัง Validation ของผู้ใช้คนเดียวกัน
- นับรายการใหม่ (cold-start) ใน Validation/Test
- ผล: ไม่พบ leakage และไม่มี cold-start item (ผลดีของ 5-core)

Part 1: เรียกใช้ Pipeline

```
K_CORE = 5
log_step("0 raw", ratings)
clean = step_clean(ratings)
core = step_kcore(clean, K_CORE, K_CORE)
core_idx, R_CSR = step_reindex(core)
train_df, valid_df, test_df = step_split(core_idx)
trainvalid_df = pd.concat([train_df, valid_df])
display(pd.DataFrame(DATA_CARD).round(4))
display(step_leakage_check(train_df, valid_df,
    test_df).to_frame("value"))
# หมายเหตุ: ถ้าผู้ใช้ให้คะแนนหลายเรื่องใน timestamp เดียวกัน
    (พบได้ใน MovieLens) ลำดับในวินาทีเดียวกันตัดสินด้วย item_id

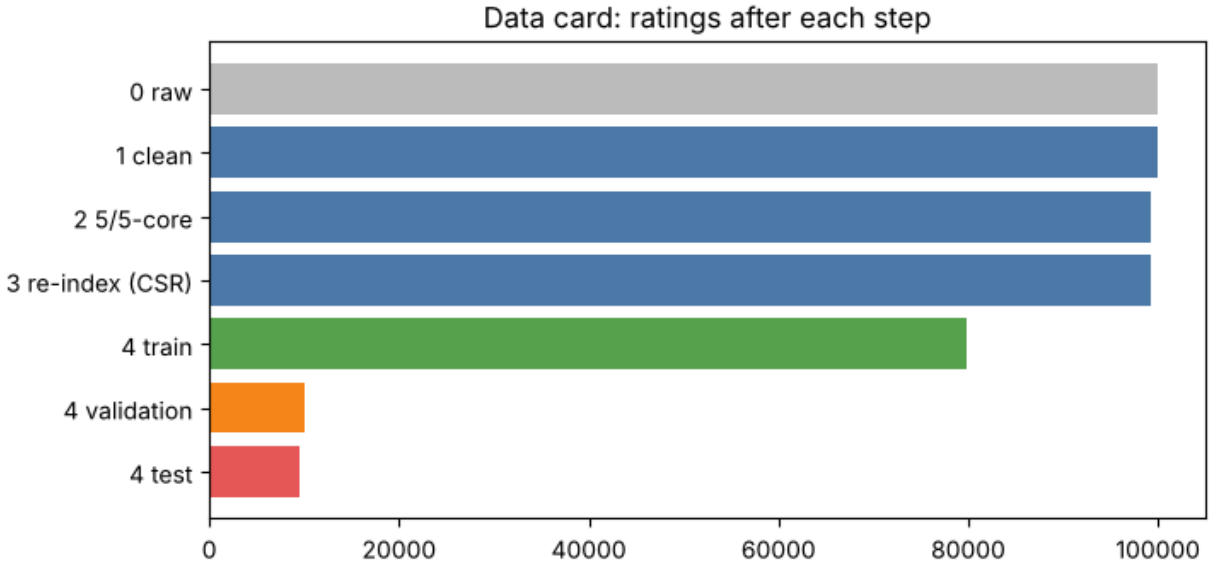
fig, ax = plt.subplots(1, 2, figsize=(14, 3.6))
dc = pd.DataFrame(DATA_CARD)
ax[0].barh(dc.step, dc.ratings, color=["#bbb", "#4C78A8",
    "#4C78A8", "#4C78A8", "#54A24B", "#F58518", "#E45756"])
ax[0].invert_yaxis(); ax[0].set_title("Data card: ratings after
    each step")
for name, part, c in [("train", train_df, "#54A24B"),
    ("validation", valid_df, "#F58518"), ("test", test_df,
    "#E45756")]:
    ax[1].hist(pd.to_datetime(part.timestamp, unit="s"), bins=50,
        alpha=.6, color=c, label=name)
ax[1].legend(); ax[1].set_title("Per-user temporal split over
    calendar time")
plt.tight_layout(); plt.show()
```

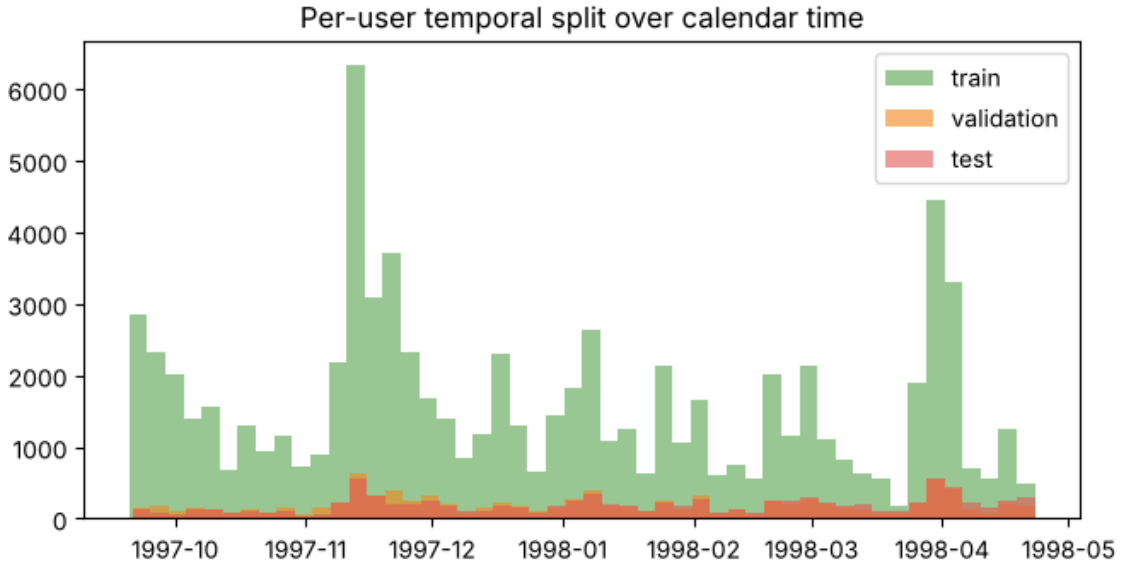
อธิบายโค้ด

หกบรรทัดได้ข้อมูลพร้อมใช้

- เรียกทีละขั้นตามลำดับ ⟶ `train_df`, `valid_df`, `test_df` และ `trainvalid_df`
- แสดง Data Card + ผลตรวจ leakage
- กราฟซ้าย: จำนวนคะแนนหลังแต่ละขั้น • ขวา: ช่วงเวลาของสามชุด
- ขนาด: Train 79,790 • Validation 9,966 • Test 9,531

Part 1: Data Card และช่วงเวลาของสามชุด





- ซ้าย: 100,000 ⟶ 99,287 หลัง 5-core แล้วแบ่งเป็น Train/Validation/Test
- ขวา: แบ่งตามเวลา **ต่อผู้ใช้** ทำให้สามชุดซ้อนกันบนปฏิทิน แต่ภายในผู้ใช้แต่ละคน Test มาทีหลังเสมอ

Part 2: Core Functions ที่เพิ่มจาก LAB 2

```python
def evaluate_rating_df(model, df):
    preds = model.algo.test(df_to_testset(df)); return rating_metrics(preds)

def evaluate_lists(lists, ts_, test_df_, k=K):
    """ประเมินรายการแนะนำที่สร้างเสร็จแล้ว (ใช้กับ pipeline ที่มีการ
        re-rank)"""
    rel_by_user = test_df_[test_df_.rating >=
        LIKE_T].groupby("user_id")["item_id"].apply(set).to_dict()
    pop = {ts_.to_raw_iid(i): len(v) for i, v in ts_.ir.items()}; rows =
        []; all_recs = []
    for uid, top in lists.items():
        rel = rel_by_user.get(uid, set())
        if not rel: continue
        rows.append({f"Precision@{k}": precision_at_k(top, rel, k),
            f"Recall@{k}": recall_at_k(top, rel, k),
                    f"nDCG@{k}": ndcg_at_k(top, rel, k), f"HR@{k}":
                        hr_at_k(top, rel, k), "ILD": ild(top),
                    "Novelty": novelty(top, pop, ts_.n_users), "ARP":
                        np.mean([pop.get(i, 0) for i in top])})
        all_recs += top
    s = pd.DataFrame(rows).mean()
    s["Coverage"] = coverage(all_recs, ts_.n_items); s["Personalization"] =
        personalization(list(lists.values()))
    return s

TS_TRAIN = df_to_trainset(train_df)
TS_TRAINVALID = df_to_trainset(trainvalid_df)
USERS_VALID = eval_users(valid_df, TS_TRAIN, n=N_EVAL_USERS)
USERS_TEST = eval_users(test_df, TS_TRAINVALID, n=N_EVAL_USERS)
print(f"Train={TS_TRAIN.n_ratings}  Valid={len(valid_df)}
    Test={len(test_df)} | eval users valid={len(USERS_VALID)}
    test={len(USERS_TEST)}")
```

อธิบายโค้ด

ประเมินรายการที่สร้างเสร็จแล้ว

- `SurpriseModel`, `EASEModel`, metric และ `evaluate_ranking` ใช้ code เดียวกับ LAB 2
- `evaluate_rating_df`: RMSE/MAE/FCP ของ DataFrame ใด ๆ
- `evaluate_lists`: วัด P/R/nDCG/HR + ILD, Novelty, Coverage จาก **รายการที่ re-rank แล้ว** (Part 6)
- `TS_TRAIN` ใช้ปรับจูน · `TS_TRAINVALID` ใช้ refit · ผู้ใช้ประเมิน 300 คน

Part 3: ระบบฐานก่อนปรับจูน

```
DEFAULT_FACTORIES = {
    "Popularity":       (lambda: PopularityModel(), "Baseline"),
    "BaselineOnly":     (lambda: SurpriseModel("BaselineOnly",
        lambda: BaselineOnly(verbose=False), "Baseline"),
        "Baseline"),
    "UserKNN-Means":    (lambda: SurpriseModel("UserKNN-Means",
        lambda: KNNWithMeans(verbose=False), "Memory-based"),
        "Memory-based"),
    "ItemKNN-Baseline": (lambda:
        SurpriseModel("ItemKNN-Baseline", lambda:
        KNNBaseline(sim_options={"name": "pearson_baseline",
        "user_based": False}, verbose=False), "Memory-based"),
        "Memory-based"),
    "SVD":              (lambda: SurpriseModel("SVD", lambda:
        SVD(random_state=SEED), "Latent factor"), "Latent
        factor"),
    "SVD++":            (lambda: SurpriseModel("SVD++", lambda:
        SVDpp(n_factors=20, random_state=SEED), "Latent factor"),
        "Latent factor"),
    "NMF":              (lambda: SurpriseModel("NMF", lambda:
        NMF(random_state=SEED), "Latent factor"), "Latent
        factor"),
    "EASE":             (lambda: EASEModel(lam=500.0),
        "Implicit"),
```

อธิบายโค้ด

ค่า default ของแต่ละอัลกอริทึม

- 8 โมเดล: Popularity, BaselineOnly, UserKNN-Means, ItemKNN-Baseline, SVD, SVD++, NMF, EASE
- เก็บเป็น factory (`lambda`) ⟶ สร้างใหม่ได้ทุกครั้ง
- ค่าเริ่มต้นของ Surprise เช่น SVD f = 100, reg 0.02, 20 รอบ
- ใช้เป็นจุดอ้างอิงว่า **การปรับจูนช่วยได้จริงหรือไม่**

Part 3: ผลระบบฐานบน Validation (ฝึกบน Train)

โมเดล	RMSE	MAE	HR@10	nDCG@10	MRR	Coverage
EASE (λ=500)	—	—	0.4133	**0.1237**	0.2111	0.1735
Popularity	—	—	0.2067	0.0487	0.1024	0.0482
SVD	0.9624	0.7576	0.1767	0.0437	0.0918	0.1468
SVD++	**0.9435**	0.7425	0.1700	0.0367	0.0850	0.1490
BaselineOnly	0.9767	0.7747	0.1533	0.0308	0.0567	0.0326
ItemKNN-Baseline	0.9593	0.7517	0.1467	0.0233	0.0561	0.2809
NMF	0.9956	0.7837	0.0600	0.0081	0.0250	0.1698
UserKNN-Means	0.9789	0.7729	0.0500	0.0056	0.0227	0.0556

จุดเริ่มต้น: SVD++ ดีสุดด้าน RMSE • EASE ดีสุดด้าน ranking — ตรงกับ LAB 2

Part 4: สี่วิธีปรับ Hyperparameter

วิธี	เครื่องมือ	Objective	ข้อมูลที่ใช้
4.1 Grid search	`GridSearchCV` (SVD 16 configs)	RMSE, MAE (3-fold CV)	Train
4.2 Random search	`RandomizedSearchCV` (KNNBaseline 8 configs)	RMSE (3-fold CV)	Train
4.3 Hold-out tuning	loop เขียนเอง (SVD, KNN, EASE)	**RMSE และ nDCG@10**	Train → Validation
4.4 Overfitting curve	จำนวนรอบของ SVD	RMSE train vs valid	Train → Validation

ห้ามใช้ Test ในขั้นนี้ — เลือกไฮเพอร์พารามิเตอร์บน Validation แล้วฝึกใหม่บน Train+Validation (5.16)

Part 4.1: GridSearchCV — SVD

```
data_train =
    Dataset.load_from_df(train_df[["user_id",
    "item_id", "rating"]], READER)
param_grid = {"n_factors": [50, 100], "n_epochs":
    [20, 40], "lr_all": [0.005, 0.01], "reg_all":
    [0.02, 0.1]}
t0 = time.time()
gs = GridSearchCV(SVD, param_grid, measures=["rmse",
    "mae"], cv=3, n_jobs=1, joblib_verbose=0)
gs.fit(data_train)
print(f"GridSearchCV: {len(gs.cv_results['params'])}
    configs × 3 folds in {time.time()-t0:.1f}s")
print("best RMSE:", round(gs.best_score["rmse"], 4),
    gs.best_params["rmse"])
cvres = pd.DataFrame(gs.cv_results)
grid_table =
    pd.concat([pd.json_normalize(cvres["params"]),
    cvres[["mean_test_rmse", "std_test_rmse",
    "mean_test_mae", "mean_fit_time"]]], axis=1)
display(grid_table.sort_values("mean_test_rmse").head(
    8).round(4))
BEST_SVD_RMSE = gs.best_params["rmse"]
```

อธิบายโค้ด

ลองทุกชุดค่าแบบ 3-fold CV

- **param_grid**: n_factors × n_epochs × lr_all × reg_all = 2×2×2×2 = 16 configs × 3 folds (≈ 20 วินาที)
- **gs.best_params["rmse"]** = f 100, 40 รอบ, lr 0.01, reg 0.1 ⟶ RMSE 0.9225
- **cv_results** แปลงเป็นตาราง เรียงตาม RMSE
- reg 0.1 ชนะทุกชุด ⟶ regularization สำคัญกว่า n_factors

Part 4.2: RandomizedSearchCV — KNNBaseline

```
param_dist = {"k": [20, 40, 60, 80], "min_k": [1, 3,
    5], "verbose": [False],
              "sim_options": {"name":
                  ["pearson_baseline", "msd",
                  "cosine"], "user_based": [False],
                  "shrinkage": [50, 100]}}
t0 = time.time()
rs = RandomizedSearchCV(KNNBaseline, param_dist,
    n_iter=8, measures=["rmse"], cv=3,
    random_state=SEED, n_jobs=1)
rs.fit(data_train)
print(f"RandomizedSearchCV: 8 configs × 3 folds in
    {time.time()-t0:.1f}s")
print("best RMSE:", round(rs.best_score["rmse"], 4),
    rs.best_params["rmse"])
rsres = pd.DataFrame(rs.cv_results)
display(pd.concat([pd.json_normalize(rsres["params"]),
    rsres[["mean_test_rmse", "std_test_rmse",
    "mean_fit_time"]]], axis=1)
        .sort_values("mean_test_rmse").round(4))
BEST_KNN_RMSE = rs.best_params["rmse"]
```

อธิบายโค้ด

สุ่มบางชุดจากช่วงค่ากว้าง

- สุ่ม 8 จาก 4×3×3×2 = 72 ชุด ⟶ ประหยัดเวลา
- **sim_options** ซ้อนเป็น dict ได้: ชื่อความคล้าย + shrinkage
- ผล: k 60, min_k 5, pearson_baseline, shrinkage 50 ⟶ RMSE 0.931
- pearson_baseline ชนะ cosine/msd ⟶ การลบค่าเอนเอียงก่อนวัดความคล้ายช่วย

Part 4.3: Hold-out tuning บน Validation

```
N_TUNE_USERS = 200
USERS_TUNE = USERS_VALID[:N_TUNE_USERS]
tune_rows = []
def tune(model_name, factory, params):
    t0 = time.time(); m = factory(); m.fit(TS_TRAIN)
    r = evaluate_rating_df(m, valid_df)["RMSE"] if m.rating_model else
        np.nan
    s, _ = evaluate_ranking(m, TS_TRAIN, valid_df, USERS_TUNE)
    tune_rows.append({"Model": model_name, **params, "RMSE": r,
        f"nDCG@{K}": s[f"nDCG@{K}"], f"HR@{K}": s[f"HR@{K}"],
                      "Coverage": s["Coverage"], "sec": time.time() - t0})

# biased=False: ตัด b_u, b_i ออก → ไม่ดันรายการคะแนนเฉลี่ยสูงแต่มีคนดูน้อย
for biased in [True, False]:
    for nf in [50, 100, 200]:
        for reg in [0.02, 0.1]:
            tune("SVD", lambda: SurpriseModel("SVD", lambda:
                SVD(n_factors=nf, reg_all=reg, biased=biased, n_epochs=20,
                random_state=SEED), "Latent factor"),
                 {"biased": biased, "n_factors": nf, "reg_all": reg})
for k_ in [10, 40]:
    for ms in [1, 10]:
        tune("ItemKNN-Baseline", lambda: SurpriseModel("ItemKNN-Baseline",
            lambda: KNNBaseline(
            k=k_, sim_options={"name": "pearson_baseline", "user_based":
                False, "min_support": ms, "shrinkage": 100},
                verbose=False), "Memory-based"),
             {"k": k_, "min_support": ms})
for lam in [50, 200, 500, 1000, 3000]:
    tune("EASE", lambda: EASEModel(lam=lam), {"lambda": lam})
tune_results = pd.DataFrame(tune_rows)
```

อธิบายโค้ด

วัดสอง objective พร้อมกัน

- `tune()` ฝึกบน Train วัด RMSE และ nDCG@10 บน Validation (200 ผู้ใช้)
- SVD: biased × n_factors × reg = 12 ชุด • KNN: k × min_support = 4 ชุด • EASE: λ 5 ค่า
- `biased=False` ตัด b_u, b_i ออกจาก SVD
- `lambda` ใน loop ถูกเรียกทันที จึงจับค่าตัวแปรถูกต้อง

Part 4.3: ผล Hold-out tuning บน Validation (บางส่วน)

โมเดล	config	RMSE	nDCG@10	HR@10	Coverage
SVD	biased, f=50, reg=0.02	**0.9569**	0.0385	0.185	0.092
SVD	biased, f=100, reg=0.02	0.9624	0.0421	0.185	0.129
SVD	no bias, f=50, reg=0.02	0.9587	0.0584	0.245	0.135
SVD	no bias, f=200, reg=0.02	0.9931	**0.0749**	0.305	0.187
ItemKNN-Baseline	k=10, min_support=1	0.9775	**0.0264**	0.165	0.286
ItemKNN-Baseline	k=40, min_support=1	**0.9593**	0.0238	0.160	0.251
ItemKNN-Baseline	k=40, min_support=10	0.9680	0.0136	0.110	0.258
EASE	λ = 50	—	**0.1457**	0.480	0.242
EASE	λ = 500 / 3000	—	0.1361 / 0.1103	0.435 / 0.390	0.165 / 0.108

config ที่ดีที่สุดสำหรับ RMSE กับ nDCG ต่างกัน: SVD biased → RMSE ดี • SVD no bias → nDCG ดีขึ้นเกือบ 2 เท่า

Part 4.3: เลือก config ตาม objective

```
best_by = {}
for name, g in tune_results.groupby("Model"):
    pcols = [c for c in g.columns if c not in
        ("Model", "RMSE", f"nDCG@{K}", f"HR@{K}",
        "Coverage", "sec") and g[c].notna().any()]
    best_r = g.loc[g.RMSE.idxmin(), pcols].to_dict()
        if g.RMSE.notna().any() else None
    best_n = g.loc[g[f"nDCG@{K}"].idxmax(),
        pcols].to_dict()
    best_by[name] = {"best for RMSE": best_r, f"best
        for nDCG@{K}": best_n}
display(pd.DataFrame(best_by).T)
```

อธิบายโค้ด

best for RMSE vs best for nDCG

- แยกตามโมเดล หาแถวที่ RMSE ต่ำสุด และแถวที่ nDCG สูงสุด
- เก็บเฉพาะคอลัมน์ไฮเพอร์พารามิเตอร์ที่โมเดลนั้นใช้จริง
- SVD: RMSE → biased f 50 · nDCG → no bias f 200
- KNN: RMSE → k 40 · nDCG → k 10 · EASE: λ 50

Part 4.4: Overfitting curve ของ SVD

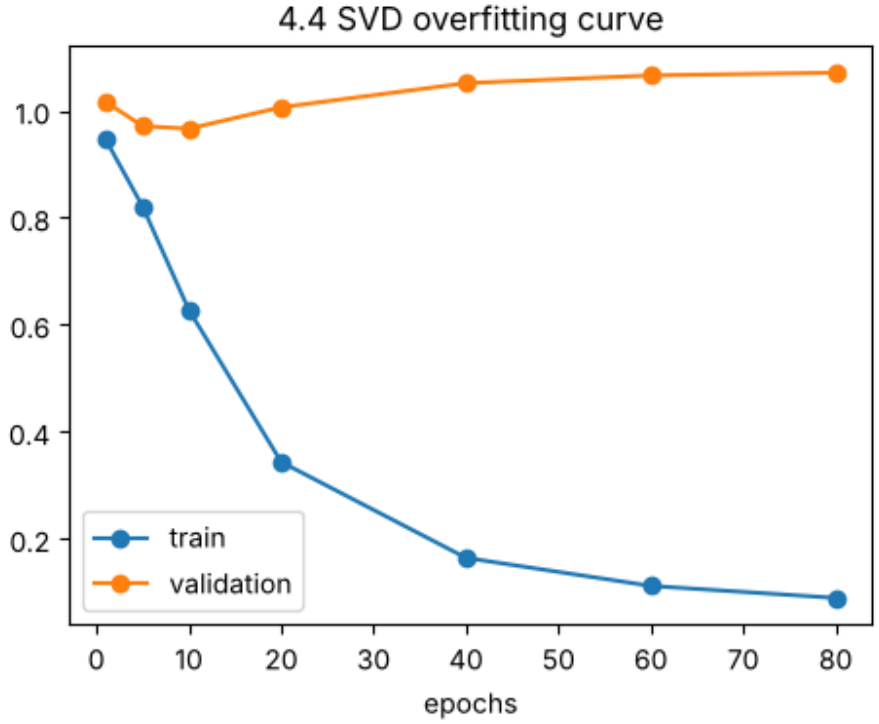
```
curve = []
for ep in [1, 5, 10, 20, 40, 60, 80]:
    a = SVD(n_factors=100, n_epochs=ep, lr_all=0.01,
        reg_all=0.005, random_state=SEED).fit(TS_TRAIN)
    tr_pred = a.test(TS_TRAIN.build_testset()); va_pred
        = a.test(df_to_testset(valid_df))
    curve.append({"epochs": ep, "train RMSE":
        accuracy.rmse(tr_pred, verbose=False), "valid
        RMSE": accuracy.rmse(va_pred, verbose=False)})
curve = pd.DataFrame(curve); display(curve.round(4))
```

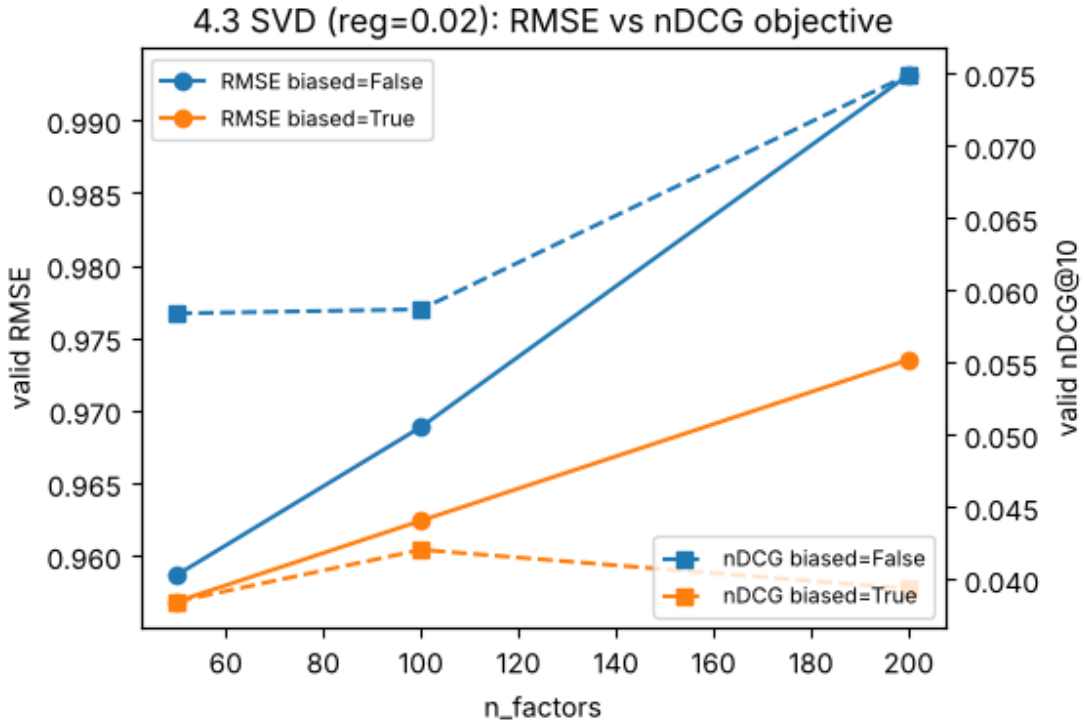
อธิบายโค้ด

ฝึกนานเกินไปก็แย่ลง (5.9.3)

- SVD f 100, lr 0.01, reg ต่ำ 0.005 แล้วเพิ่มจำนวนรอบ 1 ⟶ 80
- Train RMSE ลดจาก 0.946 เหลือ 0.090
- Validation RMSE ต่ำสุด **0.967 ที่ 10 รอบ** แล้วเพิ่มเป็น 1.071 ที่ 80 รอบ
- เป็นหลักการของ early stopping: หยุดเมื่อ Validation เริ่มแย่ลง

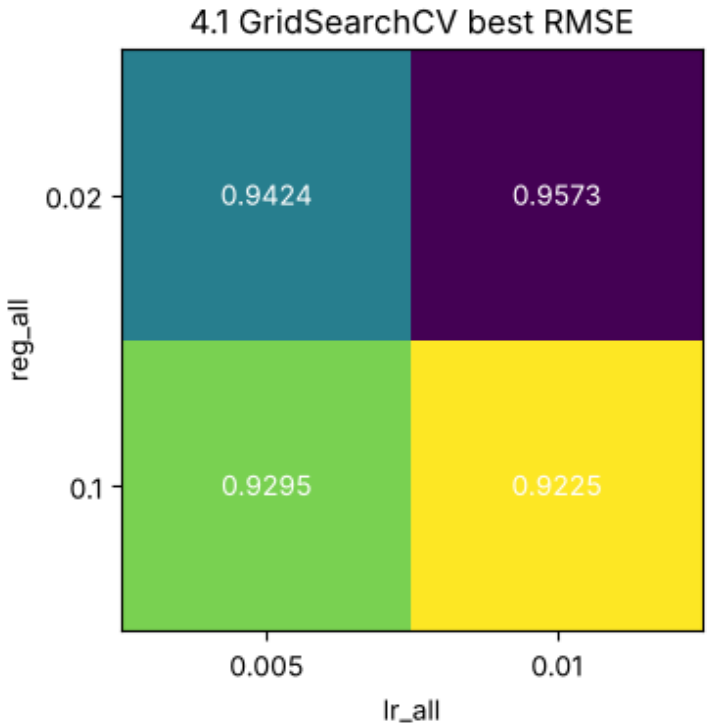
Part 4: Chart — Overfitting และ RMSE vs nDCG

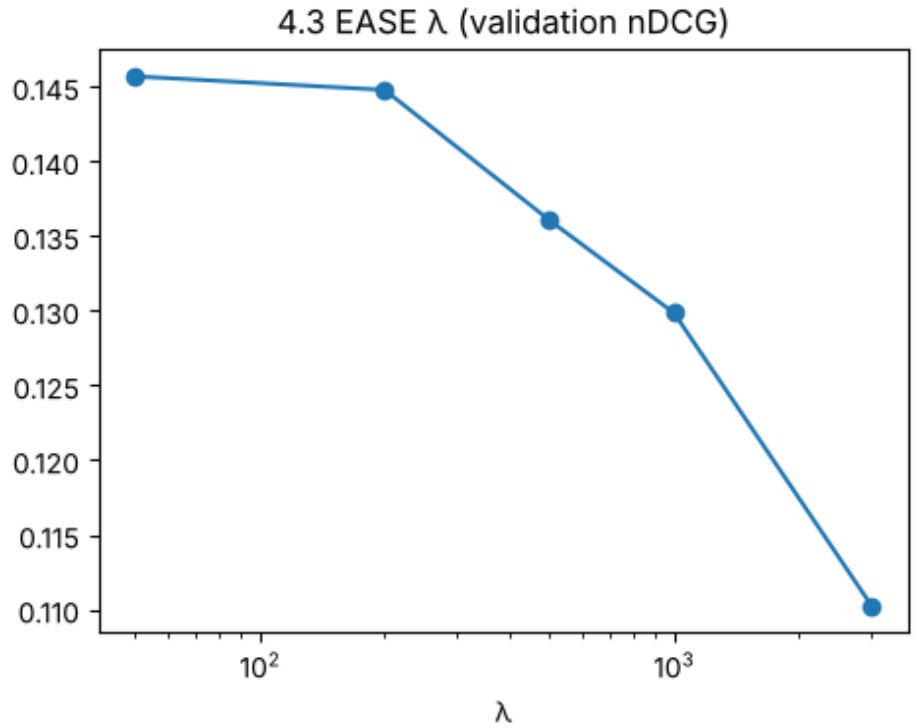




- ซ้าย: เส้น Train ลดลงเรื่อย ๆ แต่ Validation เป็นรูปตัว U ⟶ overfitting หลัง ~10 รอบ
- ขวา: SVD no bias (biased=False) ได้ nDCG สูงกว่าแบบมี bias ทุกขนาด แม้ RMSE จะแย่กว่าเล็กน้อย

Part 4: Chart — GridSearchCV และ EASE λ





- ซ้าย: heatmap RMSE ของ GridSearchCV — reg 0.1 + lr 0.01 ดีที่สุด
- ขวา: EASE λ เล็กลง $\longrightarrow$ nDCG สูงขึ้น บนข้อมูลชุดนี้ (λ = 50 ดีที่สุดในช่วงที่ลอง)

Part 5: ดึง config ที่ดีที่สุด

```
def p_int(d, k): return int(d[k]) if d and k in d and
    pd.notna(d[k]) else None
svd_ndcg = best_by["SVD"][f"best for nDCG@{K}"];
    knn_ndcg = best_by["ItemKNN-Baseline"][f"best for
    nDCG@{K}"]
ease_lam = float(best_by["EASE"][f"best for
    nDCG@{K}"]["lambda"])
knn_rs = dict(BEST_KNN_RMSE); knn_rs_sim =
    knn_rs.pop("sim_options"); knn_rs.pop("verbose",
    None)

FINAL_FACTORIES = {
    "Popularity": lambda: PopularityModel(),
    "BaselineOnly":
        DEFAULT_FACTORIES["BaselineOnly"][0],
    "ItemKNN-Baseline (default)":
        DEFAULT_FACTORIES["ItemKNN-Baseline"][0],
```

อธิบายโค้ด

จากผลปรับจูนสู่โมเดลสุดท้าย

- `best_by` (hold-out) ⟶ SVD/KNN ที่ดีที่สุดด้าน nDCG และ λ ของ EASE
- `BEST_SVD_RMSE` / `BEST_KNN_RMSE` (CV) ⟶ ตัวที่ดีที่สุดด้าน RMSE
- `p_int` แปลงค่าจากตาราง (float/NaN) เป็น int อย่างปลอดภัย
- ตัด `verbose` และ `sim_options` ออกจาก dict ก่อนส่งให้ KNNBaseline

Part 5: FINAL FACTORIES

```
    "ItemKNN-Baseline (tuned RMSE)": lambda:
        SurpriseModel("ItemKNN-Baseline (tuned RMSE)", lambda:
        KNNBaseline(**knn_rs, sim_options=knn_rs_sim, verbose=False),
        "Memory-based"),
    "ItemKNN-Baseline (tuned nDCG)": lambda:
        SurpriseModel("ItemKNN-Baseline (tuned nDCG)", lambda:
        KNNBaseline(
        k=p_int(knn_ndcg, "k"), sim_options={"name":
            "pearson_baseline", "user_based": False, "min_support":
            p_int(knn_ndcg, "min_support"), "shrinkage": 100},
            verbose=False), "Memory-based"),
    "SVD (default)": DEFAULT_FACTORIES["SVD"][0],
    "SVD (tuned RMSE)": lambda: SurpriseModel("SVD (tuned RMSE)",
        lambda: SVD(**BEST_SVD_RMSE, random_state=SEED), "Latent
        factor"),
    "SVD (tuned nDCG)": lambda: SurpriseModel("SVD (tuned nDCG)",
        lambda: SVD(n_factors=p_int(svd_ndcg, "n_factors"),
        reg_all=float(svd_ndcg["reg_all"]),

                                  biased=bool(svd_ndcg["biased"]),
                                  random_state=SEED), "Latent
                                  factor"),
    "SVD++": DEFAULT_FACTORIES["SVD++"][0],
    "EASE (default λ=500)": DEFAULT_FACTORIES["EASE"][0],
    f"EASE (tuned λ={ease_lam:g})": lambda: EASEModel(lam=ease_lam),
}
FINAL_NAMES = list(FINAL_FACTORIES)
```

อธิบายโค้ด

Default vs Tuned ในชุดเดียว

- แต่ละตระกูลมีทั้ง default, tuned RMSE และ tuned nDCG
- SVD tuned nDCG ส่ง **`biased=False`** ตามผลบน Validation
- EASE ใช้ λ ที่ดีที่สุดบน Validation (50)
- รวม 11 โมเดล (Popularity, BaselineOnly เป็นระบบอ้างอิง)

Part 5: Refit บน Train+Validation แล้วทดสอบครั้งเดียว

```
FINAL, final_rows, FINAL_DETAIL, FINAL_LISTS = {}, [], {}, {}
for name, fac in FINAL_FACTORIES.items():
    t0 = time.time(); m = fac(); m.name = name;
        m.fit(TS_TRAINVALID); FINAL[name] = m
    rmet = evaluate_rating_df(m, test_df) if m.rating_model else
        {}
    s, d, lists = evaluate_ranking(m, TS_TRAINVALID, test_df,
        USERS_TEST, keep_lists=True)
    FINAL_DETAIL[name], FINAL_LISTS[name] = d, lists
    final_rows.append({"Model": name, "Family": getattr(m,
        "family", ""), "RMSE": rmet.get("RMSE", np.nan), "MAE":
        rmet.get("MAE", np.nan),
                       "FCP": rmet.get("FCP", np.nan), **{c: s[c]
                           for c in s.index if c != "n_users"},
                           "sec": time.time() - t0})
    print(f"{name:32s} done in {time.time()-t0:.1f}s")
final_results = pd.DataFrame(final_rows)
ACC_COLS = ["RMSE", "MAE", f"Precision@{K}", f"Recall@{K}",
    f"F1@{K}", f"HR@{K}", "MRR", f"MAP@{K}", f"nDCG@{K}", "AUC"]
BEYOND_COLS = ["Coverage", "Gini", "ILD", "Novelty",
    "Calibration", "Personalization", "ARP"]
print("Test — Accuracy (Rating + Ranking)")
display(final_results[["Model"] + ACC_COLS].round(4))
print("Test — Beyond-Accuracy")
display(final_results[["Model"] + BEYOND_COLS].round(4))
```

อธิบายโค้ด

Test ถูกเปิดครั้งเดียวที่นี่

- fit บน `TS_TRAINVALID` (ข้อมูลมากขึ้น 10%) แล้ววัดบน `test_df`
- Rating: RMSE, MAE, FCP • Ranking 8 metric • Beyond 7 metric
- เก็บ `FINAL_LISTS` สำหรับ Part 7 และ GUI
- ผู้ใช้ประเมินชุดเดียวกันทุกโมเดล (`USERS_TEST` 300 คน)

Part 5: ผลบน Test — Accuracy (Rating + Ranking)

โมเดล	RMSE	MAE	P@10	R@10	HR@10	MRR	nDCG@10	AUC
EASE (tuned λ=50)	—	—	0.0790	0.1901	0.4867	0.2671	**0.1631**	0.8030
EASE (default λ=500)	—	—	0.0783	0.1713	0.4533	0.2510	0.1504	0.8730
Popularity	—	—	0.0567	0.1062	0.3700	0.1832	0.0942	0.7812
SVD (tuned nDCG)	0.9931	0.7902	0.0380	0.0729	0.2700	0.1206	0.0622	0.7536
SVD (default)	0.9853	0.7817	0.0327	0.0527	0.2533	0.1118	0.0497	0.6647
SVD (tuned RMSE)	**0.9561**	0.7588	0.0250	0.0463	0.2067	0.0759	0.0375	0.6848
SVD++	0.9647	0.7612	0.0253	0.0327	0.1867	0.1013	0.0387	0.6769
ItemKNN-B (tuned RMSE)	0.9816	0.7763	0.0267	0.0407	0.2000	0.0895	0.0391	0.6728
ItemKNN-B (tuned nDCG)	0.9887	0.7792	0.0250	0.0367	0.1967	0.0772	0.0326	0.6718
BaselineOnly	1.0016	0.8002	0.0280	0.0508	0.2167	0.0681	0.0373	0.6533

SVD ปรับเพื่อ RMSE ⟶ RMSE ดีสุดแต่ nDCG ต่ำ • ปรับเพื่อ nDCG ⟶ กลับกัน • EASE ยังนำด้าน ranking

Part 5: ผลบน Test — Beyond-Accuracy

โมเดล	Coverage	Gini	ILD	Novelty	Calibration	Personal.	ARP
EASE (tuned λ=50)	0.275	0.873	0.717	2.13	**0.691**	**0.946**	241
EASE (default λ=500)	0.185	0.918	0.709	1.88	0.687	0.915	276
Popularity	0.050	0.983	0.681	1.33	0.628	0.585	**383**
SVD (tuned nDCG)	0.207	0.937	0.699	2.49	0.611	0.826	200
SVD (tuned RMSE)	0.125	0.969	0.713	3.58	0.580	0.675	150
ItemKNN-B (tuned nDCG)	**0.297**	0.894	0.728	3.57	0.577	0.910	136
BaselineOnly	0.035	0.989	0.767	2.40	0.593	0.377	207

การปรับจูนเพื่อ ranking เพิ่ม Coverage ด้วย: EASE 0.185 → 0.275 • SVD 0.143 → 0.207

Part 6: RecPipeline (1/2)

```python
class RecPipeline:
    def __init__(self, ranker_factory, rater_factory=None,
        pool=100):
        self.ranker_factory, self.rater_factory, self.pool =
            ranker_factory, rater_factory, pool
    def fit(self, df):
        self.ts = df_to_trainset(df)
        self.ranker = self.ranker_factory().fit(self.ts)
        self.rater = self.rater_factory().fit(self.ts) if
            self.rater_factory else None
        self.pop = PopularityModel().fit(self.ts); return self
    def predict_rating(self, uid, iid):
        return self.rater.predict(uid, iid) if self.rater else
            np.nan
    def _candidates(self, uid):
        if uid in self.ts._raw2inner_id_users:
            u = self.ts.to_inner_uid(uid); sc =
                self.ranker.scores(u); seen = {j for j, _ in
                self.ts.ur[u]}
        else:  # cold-start user → popularity
            sc = self.pop.scores(0); seen = set()
        order = [j for j in np.argsort(-sc, kind="stable") if j
            not in seen][:self.pool]
        return [self.ts.to_raw_iid(j) for j in order],
            np.array([sc[j] for j in order])
```

อธิบายโค้ด

รวม ranker + rater + fallback

- **fit(df)**: สร้าง trainset, ฝึก ranker (จัดอันดับ) และ rater (ทำนายคะแนน) แยกกัน + Popularity
- **predict_rating**: ใช้ rater (SVD tuned RMSE)
- **_candidates**: ผู้ใช้เดิม ⟶ คะแนนจาก ranker ตัดเรื่องที่ดูแล้ว • ผู้ใช้ใหม่ ⟶ Popularity (cold start)
- เก็บผู้สมัคร 100 อันดับแรก (**pool**) ไว้ re-rank

Part 6: RecPipeline (2/2) — MMR re-ranking

```
def recommend(self, uid, k=10, diversity=0.0):
    ids, sc = self._candidates(uid)
    s = (sc - sc.min()) / (sc.max() - sc.min() +
        1e-12)
    V = np.array([GVEC[i] for i in ids]); V = V /
        np.maximum(np.linalg.norm(V, axis=1,
        keepdims=True), 1e-12)
    chosen, remain = [], list(range(len(ids)))
    while remain and len(chosen) < k:
        if chosen and diversity > 0:
            sim = (V[remain] @
                V[chosen].T).max(axis=1)
            val = (1 - diversity) * s[remain] -
                diversity * sim
        else:
            val = s[remain]
        b = remain[int(np.argmax(val))];
            chosen.append(b); remain.remove(b)
    return [ids[j] for j in chosen]
```

อธิบายโค้ด

เพิ่มความหลากหลายแบบควบคุมได้

- ปรับคะแนนผู้สมัครเป็นช่วง 0–1
- เลือกทีละเรื่อง: ค่า = (1 – λ)·คะแนน – λ·ความคล้ายแนวสูงสุดกับเรื่องที่เลือกแล้ว
- λ = 0 ⟶ เรียงตามคะแนนเดิม · λ มาก ⟶ หลากหลายขึ้น
- ใช้เวกเตอร์แนว GVEC วัดความคล้าย (ไม่ต้องฝึกเพิ่ม)

Part 6: เลือกโมเดล + Diversity sweep + Save/Load

```
SurpriseModel.__getstate__ = lambda self: {k: v for k, v in
    self.__dict__.items() if k != "factory"}
RecPipeline.__getstate__ = lambda self: {k: v for k, v in
    self.__dict__.items() if not k.endswith("_factory")}

best_rank_name = final_results.sort_values(f"nDCG@{K}",
    ascending=False).Model.iloc[0]
best_rate_name =
    final_results.dropna(subset=["RMSE"]).sort_values("RMSE").Model.iloc[0]
print("Ranker =", best_rank_name, "| Rater =", best_rate_name)
PIPE = RecPipeline(FINAL_FACTORIES[best_rank_name],
    FINAL_FACTORIES[best_rate_name]).fit(trainvalid_df)

# 6.1 Diversity sweep: nDCG vs ILD vs Coverage บน Test
div_rows = []
for lam in [0.0, 0.1, 0.2, 0.3, 0.5, 0.7]:
    lists = {uid: PIPE.recommend(uid, K, lam) for uid in USERS_TEST}
    div_rows.append({"λ diversity": lam, **evaluate_lists(lists, PIPE.ts,
        test_df).to_dict()})
div_results = pd.DataFrame(div_rows); display(div_results.round(4))

# 6.2 Save / Load (พร้อมนำไปใช้งาน)
with open("rec_pipeline.pkl", "wb") as f: pickle.dump(PIPE, f)
with open("rec_pipeline.pkl", "rb") as f: PIPE2 = pickle.load(f)
uid0 = USERS_TEST[0]
print("Saved & reloaded. Same output:", PIPE.recommend(uid0, 5) ==
    PIPE2.recommend(uid0, 5))
print("Cold-start user 'new_user' →", [TITLE[i] for i in
    PIPE.recommend("new_user", 5)])
```

อธิบายโค้ด

จากการทดลองสู่การใช้งาน

- ranker = nDCG สูงสุดบน Test (EASE λ 50) • rater = RMSE ต่ำสุด (SVD tuned RMSE)
- `__getstate__` ตัด `lambda` ออก ⟶ pickle ได้
- Sweep λ: 0 ⟶ 0.7 วัด nDCG, ILD, Coverage ด้วย `evaluate_lists`
- บันทึก `rec_pipeline.pkl` แล้วโหลดกลับ ⟶ ผลเหมือนเดิม (True)

Part 6: Trade-off ของ MMR re-ranking บน Test

λ diversity	nDCG@10	HR@10	ILD	Novelty	Coverage
0.0	**0.1631**	0.4867	0.7167	2.126	0.2750
0.1	0.1617	0.4833	0.7470	2.126	0.2750
0.2	0.1616	0.4867	0.7768	2.137	0.2728
0.3	0.1552	0.4700	0.8196	2.178	0.2698
0.5	0.1440	0.4367	0.9045	2.381	0.2676
0.7	0.1328	0.3967	**0.9305**	**2.501**	0.2743

λ = 0.2–0.3: ILD เพิ่ม 9–14% โดย nDCG ลดเพียง 1–5% $\longrightarrow$ จุดสมดุลที่เหมาะกับการใช้งานจริง

Part 7: Output บน Validation และ Test (Rating)

```
def short(iid, n=28):
    t = TITLE.get(iid, iid); return t if len(t) <= n else t[:n-1] +
        "…"
rel_cnt = test_df[test_df.rating >= LIKE_T].groupby("user_id").size()
SAMPLE_USERS = [u for u in rel_cnt.sort_values(ascending=False).index
    if u in USERS_TEST and u in set(valid_df.user_id)][:3]
UINFO = users.set_index("user_id")

uid = SAMPLE_USERS[0]
print(f"7.1 Validation — ผู้ใช้ {uid} ({UINFO.loc[uid].age} ปี,
    {UINFO.loc[uid].gender}, {UINFO.loc[uid].occupation})")
sub = valid_df[valid_df.user_id == uid].head(8)
tab = pd.DataFrame({"movie": [short(i) for i in sub.item_id], "true":
    sub.rating.values})
for name, m in DEFAULT_FIT.items():
    if m.rating_model: tab[name] = [m.predict(uid, i) for i in
        sub.item_id]
display(tab.round(2))

print(f"7.2 Test — ผู้ใช้ {uid}")
sub = test_df[test_df.user_id == uid].head(8)
tab = pd.DataFrame({"movie": [short(i) for i in sub.item_id], "true":
    sub.rating.values})
for name, m in FINAL.items():
    if m.rating_model: tab[name] = [m.predict(uid, i) for i in
        sub.item_id]
tab["Pipeline"] = [PIPE.predict_rating(uid, i) for i in sub.item_id]
display(tab.round(2))
```

อธิบายโค้ด

ผู้ใช้คนเดียวกัน สองช่วงเวลา

- เลือกผู้ใช้ที่มี relevant ใน Test มากที่สุดและอยู่ใน Validation ด้วย
- 7.1 Validation: โมเดล **default** (ฝึกบน Train) ทำนาย 8 คะแนน
- 7.2 Test: โมเดล **tuned** (ฝึกบน Train+Validation) + ค่าทำนายของ Pipeline
- ผู้ใช้ 416 (20 ปี, หญิง, นักศึกษา)

Part 7: ตัวอย่าง Output (Rating บน Test) — ผู้ใช้ 416

ภาพยนตร์	จริง	Baseline	KNN tuned	SVD default	SVD tuned RMSE	SVD++	Pipeline
Vertigo (1958)	5	4.52	4.47	4.45	4.80	4.51	4.80
Waiting for Guffman	5	4.23	4.45	4.42	4.68	4.72	4.68
Sound of Music	5	4.04	4.10	4.81	4.84	5.00	4.84
12 Angry Men (1957)	5	4.56	4.57	4.87	4.97	4.88	4.97
North by Northwest	5	4.55	4.48	4.67	4.87	4.37	4.87
Dumbo (1941)	5	3.73	3.69	3.91	4.04	4.00	4.04
Robin Hood: Prince of Thieves	5	3.54	3.90	3.98	4.22	4.04	4.22

SVD ที่ปรับเพื่อ RMSE ใกล้ค่าจริงกว่า default ทุกแถว • Pipeline ใช้ SVD tuned RMSE เป็น rater จึงได้ค่าเดียวกัน

Part 7: Output บน Test (Ranking)

```
for uid in SAMPLE_USERS:
    rel = set(test_df[(test_df.user_id == uid) &
        (test_df.rating >= LIKE_T)].item_id)
    comp = pd.DataFrame({name: [("✓ " if i in rel
        else "") + short(i) for i in
        FINAL_LISTS[name][uid]] for name in
        FINAL_NAMES},
                        index=[f"#{n}" for n in
                            range(1, K+1)]).T
    comp.loc["Pipeline (λ=0.3)"] = [("✓ " if i in rel
        else "") + short(i) for i in
        PIPE.recommend(uid, K, 0.3)]
    comp["hits"] = [sum(x.startswith("✓") for x in
        row) for row in comp.values]
    print(f"\n7.3 Test ranking — ผู้ใช้ {uid}:
        relevant {len(rel)} เรื่อง"); display(comp)
```

อธิบายโค้ด

Top-10 ของทุกโมเดลรวม Pipeline

- ทุกโมเดลสุดท้าย + Pipeline (λ = 0.3) ในตารางเดียว
- ✓ = ผู้ใช้ให้ ≥ 4 ในช่วง Test
- **hits** = จำนวน ✓ ใน Top-10
- ดู 3 ผู้ใช้: 416, 276, 880

Part 7: ตัวอย่าง Output (Ranking บน Test) — ผู้ใช้ 416

โมเดล	#1	#2	#3	hits@10
EASE (tuned λ=50)	✓ Sound of Music	✓ Princess Bride	✓ Dead Poets Society	7
Pipeline (λ=0.3)	✓ Sound of Music	✓ Princess Bride	✓ Dead Poets Society	7
Popularity	✓ Titanic	✓ Princess Bride	✓ Indiana Jones...	6
SVD++	✓ Good Will Hunting	✓ Titanic	Wrong Trousers	6
SVD (tuned nDCG)	✓ Dead Poets Society	Henry V	Thin Man, The	5
SVD (tuned RMSE)	Pather Panchali	✓ Titanic	✓ Good Will Hunting	3
BaselineOnly	Close Shave, A	Casablanca	Wrong Trousers	4

ผู้ใช้ 880 ทุกโมเดลได้ 0–3 hits — ค่าเฉลี่ยซ่อนความต่างรายผู้ใช้ จึงต้องดูตัวอย่างจริงประกอบ

Part 8: ตารางสรุป + Default vs Tuned

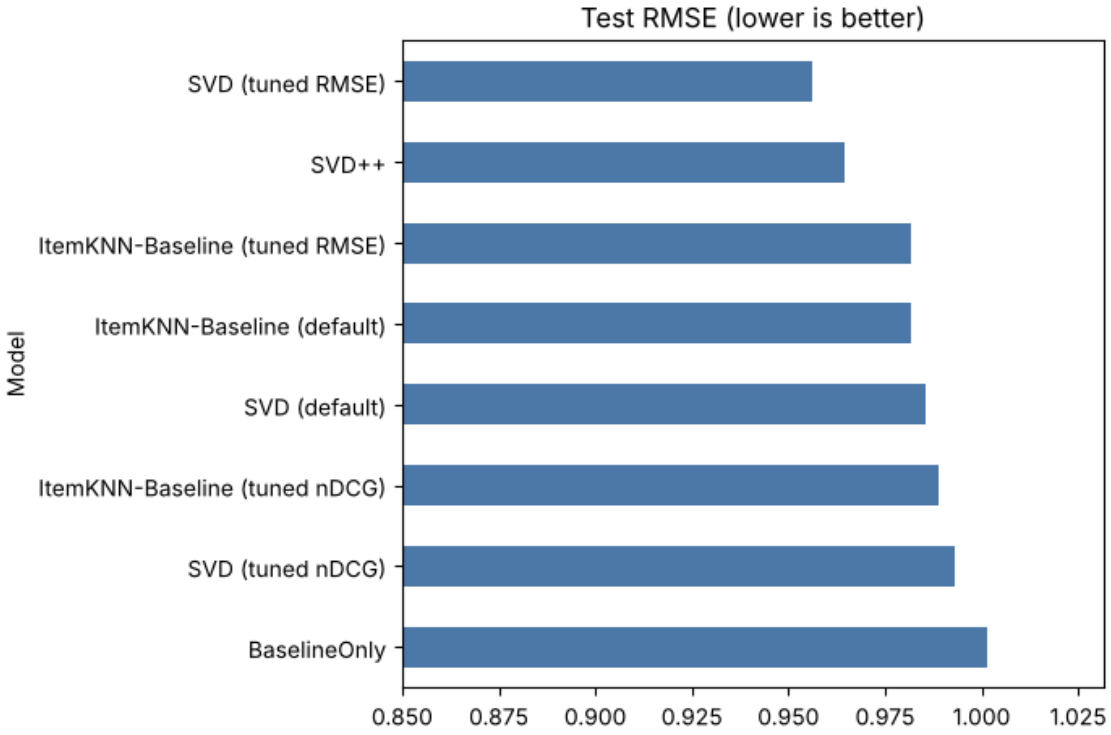
```
summary = final_results[["Model", "RMSE", "MAE",
    f"HR@{K}", "MRR", f"nDCG@{K}", "AUC"] + BEYOND_COLS]
display(summary.sort_values(f"nDCG@{K}",
    ascending=False).round(4).reset_index(drop=True))
gain = []
for fam, d_name, t_name in [("ItemKNN-Baseline",
    "ItemKNN-Baseline (default)", "ItemKNN-Baseline
    (tuned nDCG)"),
                            ("SVD", "SVD (default)", "SVD
                                (tuned nDCG)"), ("EASE",
                                "EASE (default λ=500)",
                                f"EASE (tuned
                                λ={ease_lam:g})")]:
    a, b = final_results.set_index("Model").loc[d_name],
        final_results.set_index("Model").loc[t_name]
    gain.append({"Family": fam, f"nDCG default":
        a[f"nDCG@{K}"], f"nDCG tuned": b[f"nDCG@{K}"],
                 "RMSE default": a["RMSE"], "RMSE tuned":
                     b["RMSE"], "Coverage default":
                     a["Coverage"], "Coverage tuned":
                     b["Coverage"]})
display(pd.DataFrame(gain).round(4))
```

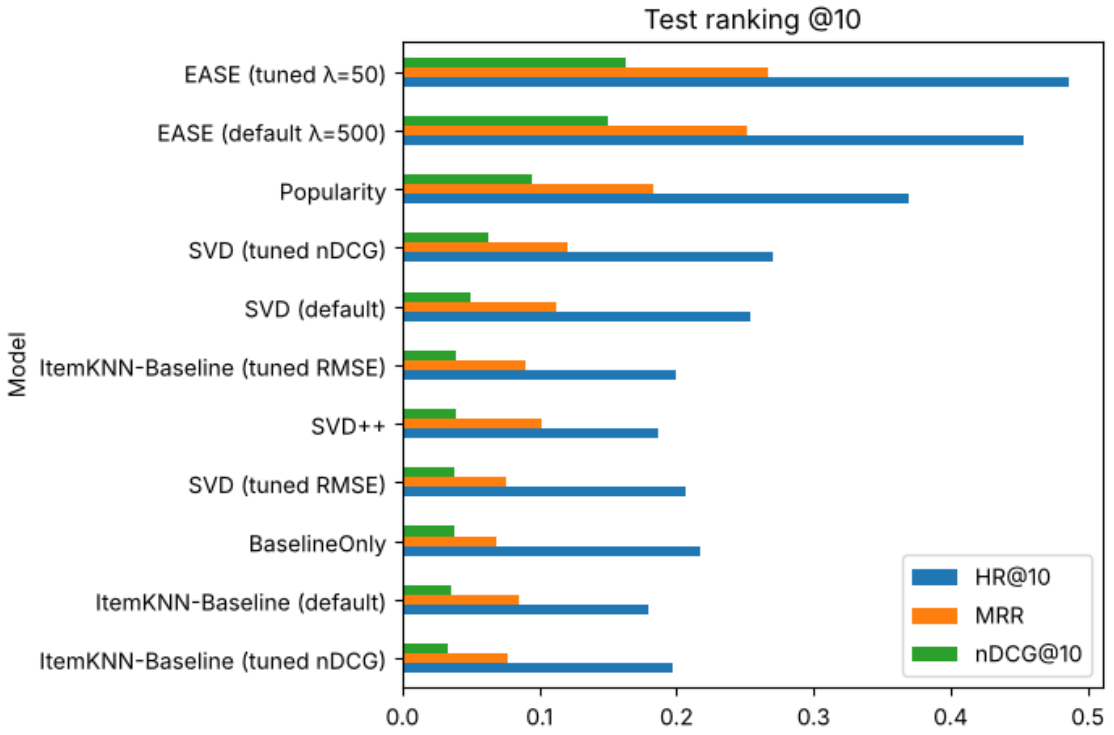
อธิบายโค้ด

การปรับจูนช่วยแค่ไหน

- **summary**: ทุกโมเดลบน Test เรียงตาม nDCG
- **gain**: เทียบ default กับ tuned nDCG ของสามตระกูล
- SVD: nDCG 0.050 → **0.062** (+25%), Coverage 0.143 → 0.207
- EASE: 0.150 → **0.163** • ItemKNN: 0.036 → 0.033 (ไม่ดีขึ้นบน Test)

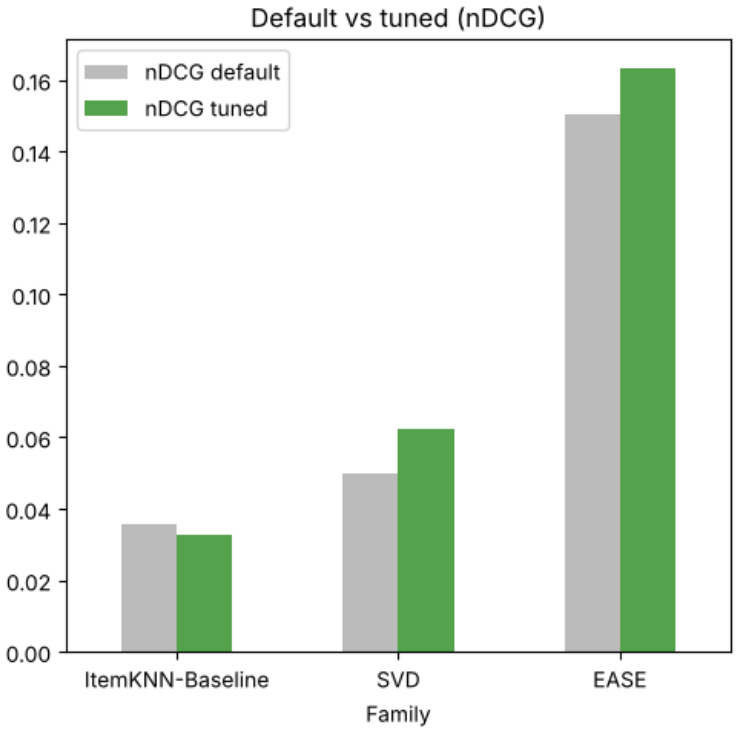
Part 8: Chart — Rating และ Ranking บน Test

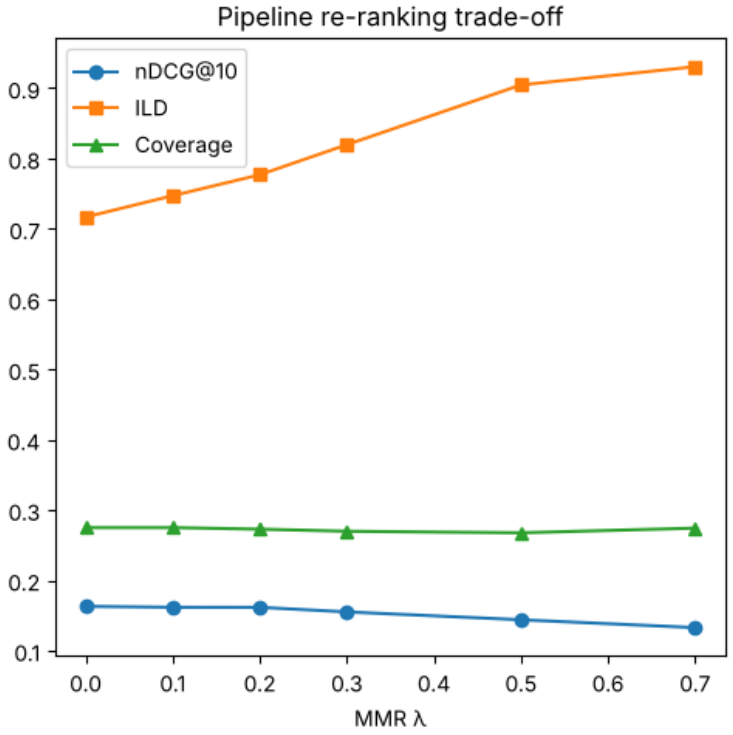




- ซ้าย: RMSE — SVD tuned RMSE ดีที่สุด (0.956) รองลงมาคือ SVD++
- ขวา: HR/MRR/nDCG@10 — EASE ทั้งสองแบบนำ ตามด้วย Popularity และ SVD tuned nDCG

Part 8: Chart — Default vs Tuned และ Trade-off





- ซ้าย: nDCG default (เทา) vs tuned (เขียว) — ดีขึ้นใน SVD และ EASE
- ขวา: MMR λ เพิ่ม $\longrightarrow$ ILD สูงขึ้นชัด ขณะที่ nDCG ลดช้า ๆ และ Coverage แทบคงที่

Part 9: GUI — ผู้ใช้เดิม

```
def gui_user(uid, model_name, k, lam):
    info = UINFO.loc[uid]; ts_ = TS_TRAINVALID; u = ts_.to_inner_uid(uid)
    hist = [(ts_.to_raw_iid(j), r) for j, r in ts_.ur[u]]
    liked = [i for i, r in hist if r >= LIKE_T]
    gtop = pd.Series([x for i in liked for x in
        GENRE[i].split("|")]).value_counts().head(5)
    fut = test_df[test_df.user_id == uid]; rel = set(fut[fut.rating >=
        LIKE_T].item_id)
    md = (f"### ผู้ใช้ {uid}\n- อายุ **{info.age}** · เพศ **{info.gender}**
        · อาชีพ **{info.occupation}**\n"
          f"- ประวัติ (Train+Validation): **{len(hist)}** คะแนน · เฉลี่ย
              **{np.mean([r for _, r in hist]):.2f}** · ชอบ
              **{len(liked)}**เรื่อง\n"
          f"- แนวที่ชอบ: {', '.join(f'{a} ({b})' for a, b in
              gtop.items())}\n- Test (อนาคต): {len(fut)} คะแนน, ชอบ
              {len(rel)} เรื่อง")
    h = pd.DataFrame([{"Movie": TITLE[i], "Genre": GENRE[i], "Rating": r}
        for i, r in hist]).sort_values("Rating", ascending=False).head(15)
    pipe = PIPE if model_name == "Pipeline (best)" else RecPipeline(lambda:
        FINAL[model_name], None)
    if pipe is not PIPE:
        pipe.ts, pipe.ranker, pipe.pop = TS_TRAINVALID, FINAL[model_name],
            PIPE.pop
    recs = pipe.recommend(uid, int(k), float(lam))
    pop = {ts_.to_raw_iid(i): len(v) for i, v in ts_.ir.items()}
    rec = pd.DataFrame([{"Rank": n, "Movie": TITLE[i], "Year": YEAR.get(i),
        "Genre": GENRE[i],
                         "Pred. rating": round(PIPE.predict_rating(uid, i),
                             2), "#ratings": pop.get(i, 0),
                         "Liked in test": "✓" if i in rel else ""} for n, i
                             in enumerate(recs, 1)])
    return md, h, rec
```

อธิบายโค้ด

ข้อมูลผู้ใช้ + รายการแนะนำ

- แสดง **อายุ เพศ อาชีพ**, จำนวนคะแนน ค่าเฉลี่ย แนวที่ชอบ และจำนวนเรื่องที่ชอบในช่วง Test
- เลือก "Pipeline (best)" หรือโมเดลสุดท้ายตัวใดก็ได้
- สไลเดอร์ Diversity (MMR **λ**) ปรับความหลากหลายได้ทันที
- ตารางแนะนำ: ปี แนว ค่าทำนาย (rater) จำนวนคะแนน และ ✓ ถ้าชอบจริง

Part 9: GUI — เทียบโมเดล

```
def gui_compare(uid, k):
    fut = test_df[test_df.user_id == uid]; rel =
        set(fut[fut.rating >= LIKE_T].item_id)
    ts_ = TS_TRAINVALID; u = ts_.to_inner_uid(uid);
        seen = {j for j, _ in ts_.ur[u]}
    out = {}
    for name, m in FINAL.items():
        sc = m.scores(u); top = [ts_.to_raw_iid(j)
            for j in np.argsort(-sc, kind="stable")
            if j not in seen][:int(k)]
        out[name] = [("✓ " if i in rel else "") +
            TITLE[i] for i in top]
    return pd.DataFrame(out, index=[f"#{n}" for n in
        range(1, int(k)+1)]).T.reset_index(names="Mode
        l")
```

อธิบายโค้ด

Top-K ของทุกโมเดลสุดท้าย

- คำนวณ `scores(u)` ของทุกโมเดลใน `FINAL` แล้วตัดเรื่องที่ดูแล้ว
- ใส่ ✓ หน้าเรื่องที่ผู้ใช้ชอบจริงในช่วง Test
- ได้ตาราง โมเดล × อันดับ เห็นความต่างของแต่ละแนวคิดทันที

Part 9: GUI — อธิบายค่าทำนาย (5.9.4)

```
def gui_explain(uid, movie):
    iid = movie.split(" | ")[0]; a = SVD_FOR_EXPLAIN.algo; ts_
        = TS_TRAINVALID
    true = core[(core.user_id == uid) & (core.item_id ==
        iid)].rating
    if uid not in ts_._raw2inner_id_users or iid not in
        ts_._raw2inner_id_items:
        return "ผู้ใช้หรือภาพยนตร์ไม่อยู่ในข้อมูลฝึก (cold
            start)", pd.DataFrame()
    u, i = ts_.to_inner_uid(uid), ts_.to_inner_iid(iid)
    parts = {"μ (global mean)": ts_.global_mean, "b_u (user
        bias)": a.bu[u], "b_i (item bias)": a.bi[i], "q_i·p_u
        (interaction)": a.qi[i] @ a.pu[u]}
    total = sum(parts.values())
    md = (f"**{TITLE[iid]}** ({GENRE[iid]}) — SVD ทำนาย
        **{np.clip(total, 1, 5):.3f}** "
          f"(จริง: {true.iloc[0] if len(true) else
              'ยังไม่เคยให้คะแนน'})")
    df = pd.DataFrame({"component": list(parts) + ["sum
        (clipped 1-5)"], "value": list(parts.values()) +
        [np.clip(total, 1, 5)]}).round(4)
    preds = pd.DataFrame([{"component": f"{n}", "value":
        round(m.predict(uid, iid), 3)} for n, m in
        FINAL.items() if m.rating_model])
    return md, pd.concat([df, preds])
```

อธิบายโค้ด

แยกส่วนประกอบของ SVD

- ค่าทำนาย = **μ** + b_u + b_i + q_i•p_u แสดงแยกทีละส่วน
- เห็นว่าคะแนนมาจากความใจดีของผู้ใช้, ความนิยมของเรื่อง หรือรสนิยมเฉพาะตัว
- ต่อท้ายด้วยค่าทำนายจากทุกโมเดลที่ทำนายคะแนนได้ + ค่าจริง
- ผู้ใช้/เรื่องไม่อยู่ในข้อมูลฝึก ⟶ แจ้งว่าเป็น cold start

Part 9: GUI — ผู้ใช้ใหม่ (Cold start)

```
def gui_newuser(liked_movies, k):
    ts_ = TS_TRAINVALID; x = np.zeros(ts_.n_items,
        dtype=np.float32); chosen = []
    for mv in liked_movies or []:
        iid = mv.split(" | ")[0]
        if iid in ts_._raw2inner_id_items:
            x[ts_.to_inner_iid(iid)] = 1;
            chosen.append(iid)
    if not chosen:
        top = [ts_.to_raw_iid(j) for j in
            np.argsort(-PIPE.pop.pop)[:int(k)]]; src =
            "Popularity (ยังไม่ได้เลือกเรื่องที่ชอบ)"
    else:
        sc = x @ EASE_FINAL.B; sc[x > 0] = -np.inf
        top = [ts_.to_raw_iid(j) for j in
            np.argsort(-sc)[:int(k)]]; src = f"EASE จาก
            {len(chosen)} เรื่องที่ชอบ"
    return src, pd.DataFrame([{"Rank": n, "Movie":
        TITLE[i], "Genre": GENRE[i]} for n, i in
        enumerate(top, 1)])
```

อธิบายโค้ด

EASE แนะนำได้ทันทีโดยไม่ต้องฝึกใหม่

- ผู้ใช้ใหม่เลือกเรื่องที่ชอบ 3–5 เรื่อง ⟶ เวกเตอร์ชอบ x
- คะแนน = x·B ของ EASE แล้วตัดเรื่องที่เลือกไว้แล้ว
- ยังไม่เลือกอะไร ⟶ Popularity (แนวทาง onboarding ในหัวข้อ 5.15.1)

Part 9: GUI — จัดหน้า (1/2)

```
with gr.Blocks(title="Chapter 5 – CF Pipeline
    (MovieLens-100K)") as demo:
    gr.Markdown("## Chapter 5 – End-to-End Collaborative
        Filtering Pipeline (MovieLens-100K, tuned models)")
    with gr.Tab("ผู้ใช้เดิม"):
        with gr.Row():
            u_dd = gr.Dropdown(USER_CHOICES,
                value=SAMPLE_USERS[0], label="User")
            m_dd = gr.Dropdown(["Pipeline (best)"] +
                FINAL_NAMES, value="Pipeline (best)",
                label="Model")
            k_sl = gr.Slider(5, 20, 10, step=1,
                label="Top-K")
            l_sl = gr.Slider(0, 0.9, 0.0, step=0.1,
                label="Diversity (MMR λ)")
        b1 = gr.Button("Recommend", variant="primary")
        md1 = gr.Markdown()
        with gr.Row():
            h1 = gr.Dataframe(label="History (top-rated)");
                r1 = gr.Dataframe(label="Recommendations")
        b1.click(gui_user, [u_dd, m_dd, k_sl, l_sl], [md1,
            h1, r1])
```

อธิบายโค้ด

แท็บผู้ใช้เดิม

- เลือก User, Model (Pipeline หรือโมเดลสุดท้าย), Top-K และ Diversity λ
- ปุ่ม Recommend $\longrightarrow$ ข้อมูลผู้ใช้ (Markdown) + History + Recommendations
- dropdown/slider ของแท็บนี้ถูกใช้ซ้ำในแท็บอื่น

Part 9: GUI — จัดหน้า (2/2)

```
    with gr.Tab("เทียบโมเดล"):
        b2 = gr.Button("Compare"); c2 =
            gr.Dataframe(label="Top-K by model (✓ =
            liked in test)")
        b2.click(gui_compare, [u_dd, k_sl], c2)
    with gr.Tab("อธิบายค่าทำนาย"):
        mv_dd = gr.Dropdown(MOVIE_CHOICES,
            value=MOVIE_CHOICES[0], label="Movie")
        b3 = gr.Button("Explain"); md3 = gr.Markdown();
            d3 = gr.Dataframe(label="SVD decomposition
            + all models")
        b3.click(gui_explain, [u_dd, mv_dd], [md3, d3])
    with gr.Tab("ผู้ใช้ใหม่ (Cold start)"):
        lk = gr.Dropdown(MOVIE_CHOICES,
            multiselect=True, label="เลือกเรื่องที่ชอบ
            (3–5เรื่อง)")
        b4 = gr.Button("Recommend for new user"); md4 =
            gr.Markdown(); d4 =
            gr.Dataframe(label="Recommendations")
        b4.click(gui_newuser, [lk, k_sl], [md4, d4])
```

```
demo.launch(debug=False)
```

อธิบายโค้ด

อีกสามแท็บ

- **เทียบโมเดล**: ใช้ User และ Top-K จากแท็บแรก
- **อธิบายค่าทำนาย**: เลือกภาพยนตร์ ⟶ ส่วนประกอบของ SVD + ทุกโมเดล
- **ผู้ใช้ใหม่**: `multiselect=True` เลือกเรื่องที่ชอบหลายเรื่อง
- แนวทางใช้: ผู้ใช้ 416 ⟶ ลอง λ = 0 กับ 0.5 แล้วดูแนวของรายการที่เปลี่ยน

สรุปแนวคิดของ Source Code LAB 3

Take-away

- Pipeline ที่ดีเริ่มจาก **ข้อมูลที่ตรวจสอบได้**: Data Card, k-core ที่รายงานค่า k, split ตามเวลา และ leakage check
- ปรับจูนบน **Validation เท่านั้น** แล้ว refit บน Train+Validation และเปิด Test ครั้งเดียว
- Objective กำหนดผล: SVD ที่ปรับเพื่อ RMSE กับเพื่อ nDCG ได้ config ต่างกันและเก่งคนละด้าน
- EASE เป็นฐานที่แข็งแรงสำหรับ ranking • MMR $\lambda \approx 0.2–0.3$ เพิ่มความหลากหลายโดยเสียความแม่นน้อย
- Pipeline ที่ใช้งานได้จริงต้องมี rater + ranker + cold-start fallback + save/load และ GUI ที่อธิบายผลได้

LAB 1 พื้นฐาน ⟶ LAB 2 เปรียบเทียบหลายโปรโตคอล ⟶ LAB 3 จากการทดลองสู่ระบบที่นำไปใช้งาน